Seoul National University

GENGFaker

Yongjun Lee, Jaehyun Choi, Jaehyeon Lee

June, 2026

Contents

1 Data Structure 1
1.1 Interval Tree structures . . . 1
1.2 Miscellaneous . . . 3

2 Combinatorics 4
2.1 Permutations . . . 4
2.2 Partitions and subsets . . . 4
2.3 Miscellaneous Sequences . . . 4

3 Math 4
3.1 Matrices . . . 4
3.2 Fast Fourier Transform . . . 5
3.3 Number Theory . . . 6
3.4 Primes . . . 6
3.5 Primality . . . 6

4 DP Optimization 8
4.1 Convex Hull Trick . . . 8
4.2 Divide and Conquer Optimization . . . 9
4.3 Monotone Queue Optimization . . . 9

5 Graph 9
5.1 Theorems . . . 9
5.2 Articulation . . . 9
5.3 Strongly Connected Component . . . 10
5.4 Biconnected Component . . . 10
5.5 Network Flow . . . 10
5.6 Minimum Distance . . . 11
5.7 Tree . . . 12

6 Geometry 12
6.1 Point . . . 12
6.2 Line . . . 13
6.3 Polygon . . . 13
6.4 Circle . . . 14

7 Strings 14
7.1 Knuth-Morris-Pratt . . . 14
7.2 Palindrome . . . 14
7.3 Trie . . . 15
7.4 AhoCorasick . . . 15
7.5 Suffix Array . . . 15
7.6 Z . . . 15

Data Structure (1)

1.1 Interval Tree structures

FenwickTree.h

Description: Fenwick Tree (Binary Indexed Tree) supports point updates and prefix queries. Useful for maintaining cumulative sums efficiently.
Time: Both operations are $\mathcal{O}(\log N)$.

13 lines

```
// 1-index
struct Fenwick {
    vector<ll> tree;

    Fenwick() = default;
    Fenwick(int sz) { tree.resize(sz); }
    void update(int i, ll dif) { while (i < tree.size()) { tree
        [i] += dif; i += (i & -i); } }
    ll query(int i) {
        ll ret = 0;
        while (i > 0) { ret += tree[i]; i -= (i & -i); }
        return ret;
    }
};
```

FenwickTree2D.h

Description: 2D Fenwick Tree supports point updates and prefix queries in a 2D grid. Useful for maintaining cumulative sums in two dimensions efficiently.
Time: Both operations are $\mathcal{O}(\log N \cdot \log M)$, where N and M are the dimensions of the grid.

20 lines

```
// 1-index
struct Fenwick2D {
    int n, m;
    vector<vector<ll>> tree;

    Fenwick2D(int n, int m): n(n), m(m) {
        tree.resize(n);
        for (int i = 0; i < n; ++i) tree[i].resize(m);
    }
    void update(int y, int x, ll dif) {
        for (int i = y; i < n; i += (i & -i))
            for (int j = x; j < m; j += (j & -j)) tree[i][j] +=
                dif;
    }
    ll query(int y, int x) {
        ll ret = 0;
        for (int i = y; i; i -= (i & -i))
            for (int j = x; j; j -= (j & -j)) ret += tree[i][j
                ];
        return ret;
    }
};
```

GoldMine.h

Description: Segment Tree implementation for solving the Gold Mine problem. Supports range queries and point updates efficiently.
Time: Both update and query operations are $\mathcal{O}(\log N)$.

61 lines

```
struct Node {
  ll lmx, rmx, mx, sm;

  Node() = default;
  Node(ll x): lmx(x), rmx(x), mx(x), sm(x) {}
};

struct SegmentTree {
  vector<ll> arr;
  vector<Node> tree;
  int sz;

  inline Node merge(const Node& l, const Node& r) {
    Node ret{};
    ret.lmx = max(l.lmx, l.sm + r.lmx);
    ret.rmx = max(r.rmx, r.sm + l.rmx);
    ret.mx = max({l.mx, r.mx, l.rmx + r.lmx});
    ret.sm = l.sm + r.sm;
    return ret;
  }
  SegmentTree() = default;
  SegmentTree(const vector<ll>& a) {
    sz = a.size();
    arr = a;
    int h = (int) ceil(log2(sz));
    int tree_size = (1 << (h + 1));
    tree = vector<Node>(tree_size);
    init(1, 0, sz - 1);
  }
  void init(int node, int start, int end) {
    if (start == end) tree[node] = Node {arr[start]};
    else {
      int mid = (start + end) / 2;
      init(node * 2, start, mid);
      init(node * 2 + 1, mid + 1, end);
      tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
    }
  }
  void update(int node, int start, int end, int index, ll val)
       {
    if (index < start || index > end) return;
    if (start == end) {
      arr[index] += val;
      tree[node] = Node {arr[index]};
      return;
    }
    int mid = (start + end) / 2;
    update(node * 2, start, mid, index, val);
    update(node * 2 + 1, mid + 1, end, index, val);
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
  }
  Node query(int node, int start, int end, int left, int right)
       {
    if (left > end || right < start) return Node {INT32_MIN};
    if (left <= start && end <= right) return tree[node];
    int mid = (start + end) / 2;
    Node l = query(node * 2, start, mid, left, right);
    Node r = query(node * 2 + 1, mid + 1, end, left, right);
    return merge(l, r);
  }
  void update(int index, ll val) { update(1, 0, sz - 1, index,
       val); }
  Node query(int left, int right) { return query(1, 0, sz - 1,
       left, right); }
};
```

LazyPropagation.h

Description: Segment Tree with Lazy Propagation for efficient range updates and queries. Supports range updates and range queries efficiently.
Time: Both update and query operations are $\mathcal{O}(\log N)$.

65 lines

```
struct LazyPropagation {
  vector<ll> arr;
```

```
vector<ll> tree;
vector<ll> lazy;
int sz;

inline ll merge(ll a, ll b) { return a + b; }
LazyPropagation() = default;
LazyPropagation(int sz): LazyPropagation(vector<ll>(sz, 0))
    {} // TODO
LazyPropagation(const vector<ll>& a) {
  sz = a.size();
  arr = a;
  int h = (int)ceil(log2(sz));
  int tree_size = (1 << (h + 1));
  tree = vector<ll>(tree_size);
      lazy = vector<ll>(tree_size);
  init(1, 0, sz - 1);
}
void init(ll node, ll start, ll end) {
  if (start == end) tree[node] = arr[start];
  else {
    int mid = (start + end) / 2;
    init(node * 2, start, mid);
    init(node * 2 + 1, mid + 1, end);
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
  }
}
void update_lazy(ll node, ll start, ll end) {
  if (lazy[node] != 0) {
    tree[node] += (end - start + 1) * lazy[node]; // TODO
    if (start != end) {
      lazy[node * 2] = merge(lazy[node * 2], lazy[node]);
      lazy[node * 2 + 1] = merge(lazy[node * 2 + 1], lazy[
          node]);
    }
    lazy[node] = 0;
  }
}
void update_range(ll node, ll start, ll end, ll left, ll
    right, ll diff) {
  update_lazy(node, start, end);
  if (left > end || right < start) return;
  if (left <= start && end <= right) {
    tree[node] += (end - start + 1) * diff; // TODO
    if (start != end) {
      lazy[node * 2] = merge(lazy[node * 2], diff); // TODO
      lazy[node * 2 + 1] = merge(lazy[node * 2 + 1], diff);
          // TODO
    }
    return;
  }
  int mid = (start + end) / 2;
  update_range(node * 2, start, mid, left, right, diff);
  update_range(node * 2 + 1, mid + 1, end, left, right, diff)
      ;
  tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
}
ll query(ll node, ll start, ll end, ll left, ll right) {
  update_lazy(node, start, end);
  if (left > end || right < start) return 0; // TODO
  if (left <= start && end <= right) return tree[node];
  int mid = (start + end) / 2;
  ll lsum = query(node * 2, start, mid, left, right);
  ll rsum = query(node * 2 + 1, mid + 1, end, left, right);
  return merge(lsum, rsum);
}
void update(ll left, ll right, ll val) { update_range(1, 0,
    sz - 1, left, right, val); }
ll query(ll left, ll right) { return query(1, 0, sz - 1, left
    , right); }
};
```

MergeSortTree.h

Description: Merge Sort Tree supports range queries to count elements satisfying a condition. Useful for answering range queries efficiently.
Time: Build $\mathcal{O}(N \log N)$, Query $\mathcal{O}(\log^2 N)$. 34 lines

```
struct MergeSortTree {
  vector<ll> arr;
  vector<vector<ll>> tree;
  int sz;

  MergeSortTree() = default;
  MergeSortTree(vector<ll>& a) {
    sz = a.size();
    arr = a;
    int h = (int)ceil(log2(sz));
    int tree_size = (1 << (h + 1));
    tree = vector<vector<ll>>(tree_size);
    init(1, 0, sz - 1);
  }
  void init(int node, int start, int end) {
    if (start == end) tree[node].push_back(arr[start]);
    else {
      int mid = (start + end) / 2;
      init(node * 2, start, mid);
      init(node * 2 + 1, mid + 1, end);
      tree[node].resize(tree[node * 2].size() + tree[node * 2 +
          1].size());
      merge(tree[node * 2].begin(), tree[node * 2].end(), tree[
          node * 2 + 1].begin(), tree[node * 2 + 1].end(),
          tree[node].begin());
    }
  }
  // # of element <= k
  ll query(int node, int start, int end, int left, int right,
      ll k) {
    if (left > end || right < start) return 0;
    if (left <= start && end <= right) return upper_bound(tree[
        node].begin(), tree[node].end(), k) - tree[node].begin
        ();
    ll lcnt = query(node * 2, start, (start + end) / 2, left,
        right, k);
    ll rcnt = query(node * 2 + 1, (start + end) / 2 + 1, end,
        left, right, k);
    return lcnt + rcnt;
  }
  ll query(int left, int right, ll k) { return query(1, 0, sz -
      1, left, right, k); }
};
```

PST.h

Description: Persistent Segment Tree supports range queries and updates with versioning. Allows access to any previous version of the segment tree.
Time: Update $\mathcal{O}(\log N)$, Query $\mathcal{O}(\log N)$. 66 lines

```
struct Node { int l, r; ll val; };

struct PST {
  vector<Node> tree;
  vector<int> root; // 1-index (0: dummy)
  int sz;

  PST() = default;
  PST(int sz): sz(sz) {
      tree.resize(1);
      root.resize(1);
  }
    // p -> c
  void newRoot(int p, int c) {
      assert(p < root.size());
      while (root.size() <= c) root.push_back(0);
      root[c] = tree.size();
    tree.push_back(Node{tree[root[p]]});
      return;
  }
  void update(int node, int start, int end, int index, ll val)
      {
    if (index < start || index > end) return;
    tree[node].val += val;
    if (start == end) return;
    int mid = (start + end) / 2;
    if (index <= mid) {
      int lidx = tree[node].l;
      tree[node].l = tree.size();
      tree.push_back(Node{tree[lidx]});
      update(tree[node].l, start, mid, index, val);
    }
    if (index > mid) {
      int ridx = tree[node].r;
      tree[node].r = tree.size();
      tree.push_back(Node{tree[ridx]});
      update(tree[node].r, mid + 1, end, index, val);
    }
  }
  ll query(int node, int start, int end, int left, int right) {
    if (left > end || right < start) return 0;
    if (left <= start && end <= right) return tree[node].val;
      int mid = (start + end) / 2;
    ll lsum = query(tree[node].l, start, mid, left, right);
    ll rsum = query(tree[node].r, mid + 1, end, left, right);
    return lsum + rsum;
  }
    int kth(int l, int r, int start, int end, ll k) {
      if (start == end) return start;
      ll tmp = tree[tree[r].l].val - tree[tree[l].l].val;
      int mid = (start + end) / 2;
      if (tmp >= k) return kth(tree[l].l, tree[r].l, start,
          mid, k);
      else return kth(tree[l].r, tree[r].r, mid + 1, end, k -
          tmp);
    }
    void update(int rt, int index, ll val) {
      assert(rt < root.size());
      update(root[rt], 0, sz - 1, index, val);
    }
  ll query(int rt, int left, int right) {
      assert(rt < root.size());
      return query(root[rt], 0, sz - 1, left, right);
    }
    int kth(int l, int r, int k) {
      assert(l - 1 < root.size() && r < root.size());
      return kth(root[l - 1], root[r], 0, sz - 1, k);
    }
};
```

SegmentTree.h

Description: Segment Tree supports range queries and point updates efficiently. Useful for maintaining cumulative sums or other associative operations.
Time: Build $\mathcal{O}(N)$, Update $\mathcal{O}(\log N)$, Query $\mathcal{O}(\log N)$. 48 lines

```
struct SegmentTree {
  vector<ll> arr;
  vector<ll> tree;
  int sz;

  ll merge(ll a, ll b) { return a + b; }
```

```
  SegmentTree() = default;
  SegmentTree(int sz): SegmentTree(vector<ll>(sz, 0)) {} // TODO
  SegmentTree(const vector<ll>& a) {
    sz = a.size();
    arr = a;
    int h = (int)ceil(log2(sz));
    int tree_size = (1 << (h + 1));
    tree = vector<ll>(tree_size);
    init(1, 0, sz - 1);
  }
  void init(int node, int start, int end) {
    if (start == end) tree[node] = arr[start];
    else {
      int mid = (start + end) / 2;
      init(node * 2, start, mid);
      init(node * 2 + 1, mid + 1, end);
      tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
    }
  }
  void update(int node, int start, int end, int index, ll val) {
    if (index < start || index > end) return;
    if (start == end) {
      arr[index] += val;
      tree[node] += val;
      return;
    }
    int mid = (start + end) / 2;
    update(node * 2, start, mid, index, val);
    update(node * 2 + 1, mid + 1, end, index, val);
    tree[node] = merge(tree[node * 2], tree[node * 2 + 1]);
  }
  ll query(int node, int start, int end, int left, int right) {
    if (left > end || right < start) return 0; // TODO
    if (left <= start && end <= right) return tree[node];
    int mid = (start + end) / 2;
    ll lsum = query(node * 2, start, mid, left, right);
    ll rsum = query(node * 2 + 1, mid + 1, end, left, right);
    return merge(lsum, rsum);
  }
  void update(int index, ll val) { update(1, 0, sz - 1, index, val); }
  ll query(int left, int right) { return query(1, 0, sz - 1, left, right); }
};
```

1.2 Miscellaneous

UnionFind.h

Description: Union-Find (Disjoint Set Union) with path compression and union by rank. Supports efficient merging and finding of connected components.
Time: $\mathcal{O}(\alpha(N))$ per operation, where α is the inverse Ackermann function.

22 lines

```
// 0-index
struct UF {
  vector<int> par, rk;

  UF(int _sz) {
    par.resize(_sz);
    rk.resize(_sz);
    for (int i = 0; i < _sz; ++i) par[i] = i;
  }
  int find(int x) {
    if (par[x] == x) return x;
    par[x] = find(par[x]);
    return par[x];
  }
  void merge(int x, int y) {
    x = find(x); y = find(y);
    if (x == y) return;
    if (rk[x] > rk[y]) swap(x, y);
    par[x] = y;
    if (rk[x] == rk[y]) ++rk[y];
  }
};
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

33 lines

```
// 0-index
struct UF {
  vector<int> par, rk;
    vector<tuple<int, int, int>> st;

    UF() = default;
  UF(int _sz) {
    par.resize(_sz);
    rk.resize(_sz);
    for (int i = 0; i < _sz; ++i) par[i] = i;
  }
    int time() { return st.size(); }
    void rollback(int t) {
        while (st.size() > t) {
            auto [v, p, r] = st.back(); st.pop_back();
            par[v] = p;
            rk[v] = r;
        }
    }
  int find(int x) {
    if (par[x] == x) return x;
    return find(par[x]);
  }
  void merge(int x, int y) {
    x = find(x); y = find(y);
    if (x == y) return;
        st.push_back({x, par[x], rk[x]});
        st.push_back({y, par[y], rk[y]});
    if (rk[x] > rk[y]) swap(x, y);
    par[x] = y;
    if (rk[x] == rk[y]) ++rk[y];
  }
} uf;
```

MO.h

Description: Mo's Algorithm for answering range queries efficiently. Sorts queries in a specific order to minimize the number of add/del operations.
Time: $\mathcal{O}\left((N+Q)\sqrt{N}\right)$, where N is the size of the array and Q is the number of queries.

29 lines

```
int N, sqrtN, M;

struct Query {
  int s, e, idx;
};

void add(int idx, bool last);

void del(int idx, bool last);

void solve() {
  vector<Query> query;
  sqrtN = sqrt(N);
  sort(query.begin(), query.end(), [](const Query& a, const Query& b) {
    if (a.s / sqrtN == b.s / sqrtN) return a.e < b.e;
    return a.s / sqrtN < b.s / sqrtN;
  });

  vector<int> ans(M);
  int s = 1, e = 1;
  add(1, true);
  for (int i = 0; i < M; ++i) {
    while (e < query[i].e) add(++e, true);
    while (e > query[i].e) del(e--, true);
    while (s < query[i].s) del(s++, false);
    while (s > query[i].s) add(--s, false);
  }
  for (int i = 0; i < M; ++i) cout << ans[i] << '\n';
}
```

OfflineDynamicConnectivity.h

Description: Offline Dynamic Connectivity using a segment tree and rollbackable Union-Find. Handles edge additions/removals and connectivity queries offline. The timeline of queries and edge updates is divided using a segment tree. A rollbackable Union-Find data structure is used to maintain connectivity information.
Time: $\mathcal{O}(Q \log Q \cdot \alpha(N))$, where Q is the number of queries/updates and N is the number of vertices.

80 lines

```
// 0-index
struct UF {
  vector<int> par, rk;
    vector<tuple<int, int, int>> st;

    UF() = default;
  UF(int _sz) {
    par.resize(_sz);
    rk.resize(_sz);
    for (int i = 0; i < _sz; ++i) par[i] = i;
  }
    int time() { return st.size(); }
    void rollback(int t) {
        while (st.size() > t) {
            auto [v, p, r] = st.back(); st.pop_back();
            par[v] = p;
            rk[v] = r;
        }
    }
  int find(int x) {
    if (par[x] == x) return x;
    return find(par[x]);
  }
  void merge(int x, int y) {
    x = find(x); y = find(y);
    if (x == y) return;
        st.push_back({x, par[x], rk[x]});
        st.push_back({y, par[y], rk[y]});
    if (rk[x] > rk[y]) swap(x, y);
    par[x] = y;
    if (rk[x] == rk[y]) ++rk[y];
  }
} uf;

struct Edge {
  int u, v, s, e;
};

vector<vector<Edge>> tree;
vector<pair<int, int>> queries;
vector<int> ans;

void update(int node, int start, int end, Edge e) {
    if (e.s > end || e.e < start) return;
```

```
    if (e.s <= start && end <= e.e) {
        tree[node].push_back(e);
        return;
    }
  int mid = (start + end) / 2;
    update(node * 2, start, mid, e);
    update(node * 2 + 1, mid + 1, end, e);
}

void dnc(int node, int start, int end) {
    int t = uf.time();
  for (auto i: tree[node]) uf.merge(i.u, i.v);
  if (start == end) {
        if (uf.find(queries[start].first) == uf.find(queries[
            start].second)) ans[start] = 1v;
  } else {
    int mid = (start + end) / 2;
    dnc(node * 2, start, mid);
    dnc(node * 2 + 1, mid + 1, end);
  }
    uf.rollback(t);
}

int N, Q;

void solve() {
    uf = UF {N};

    ans = vector<int>(Q, -1);
    int h = (int)ceil(log2(Q));
  int tree_size = (1 << (h + 1));
    tree.resize(tree_size);

    for (auto& i: bridges) update(1, 0, q - 1);

    dnc(1, 0, q - 1);
}
```

Combinatorics (2)

2.1 Permutations

Cycles Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

Derangement Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor \frac{n!}{e} \right\rceil$$

Burnside's lemma Given a group G of symmetries and a set X, the number of elements of X *up to symmetry* equals

$$\frac{1}{|G|}\sum_{g\in G}|X^g|,$$

where X^g are the elements fixed by g ($g.x = x$).

If $f(n)$ counts "configurations" (of some sort) of length n, we can ignore rotational symmetry using $G = \mathbb{Z}_n$ to get

$$g(n) = \frac{1}{n}\sum_{k=0}^{n-1} f(\gcd(n,k)) = \frac{1}{n}\sum_{k|n} f(k)\phi(n/k).$$

2.2 Partitions and subsets

2.2.1 Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0) = 1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}} (-1)^{k+1} p(n-k(3k-1)/2)$$

$$p(n) \sim 0.145/n \cdot \exp\left(2.56\sqrt{n}\right)$$

2.2.2 Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_k p^k + \ldots + n_1 p + n_0$ and $m = m_k p^k + \ldots + m_1 p + m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k} \binom{n_i}{m_i} \pmod p$.

2.3 Miscellaneous Sequences

2.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able).
$B[0,\ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty}\frac{B_k}{k!}f^{(k-1)}(m)$$
$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

2.3.2 Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\textstyle\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\ldots(x+n-1)$$

$c(8,k) = 8, 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \ldots$

2.3.3 Eulerian numbers

Number of permutations $\pi \in S_n$ in which exactly k elements are greater than the previous element. k j:s s.t. $\pi(j) > \pi(j+1)$, $k+1$ j:s s.t. $\pi(j) \geq j$, k j:s s.t. $\pi(j) > j$.

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$

$$E(n,0) = E(n,n-1) = 1$$

$$E(n,k) = \sum_{j=0}^{k}(-1)^j\binom{n+1}{j}(k+1-j)^n$$

2.3.4 Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$

$$S(n,1) = S(n,n) = 1$$

$$S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$$

2.3.5 Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \ldots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod p$$

2.3.6 Labeled unrooted trees

on n vertices: n^{n-2}
on k existing trees of size n_i: $n_1 n_2 \cdots n_k n^{k-2}$
with degrees d_i: $(n-2)!/((d_1-1)!\cdots(d_n-1)!)$

2.3.7 Catalan numbers

$$C_n = \frac{1}{n+1}\binom{2n}{n} = \binom{2n}{n} - \binom{2n}{n+1} = \frac{(2n)!}{(n+1)!n!}$$

$$C_0 = 1,\ C_{n+1} = \frac{2(2n+1)}{n+2}C_n,\ C_{n+1} = \sum C_i C_{n-i}$$

$C_n = 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, \ldots$

- sub-diagonal monotone paths in an $n \times n$ grid.
- strings with n pairs of parenthesis, correctly nested.
- binary trees with with $n+1$ leaves (0 or 2 children).
- ordered trees with $n+1$ vertices.
- ways a convex polygon with $n+2$ sides can be cut into triangles by connecting vertices with straight lines.
- permutations of $[n]$ with no 3-term increasing subseq.

2.3.8 Narayana numbers

The number of Dyck paths of length $2n$ with exactly k peaks.

$$N(n,k) = \frac{1}{n}\binom{n}{k}\binom{n}{k-1}$$

$$\sum_{k=1}^{n} N(n,k) = C_n$$

Math (3)

3.1 Matrices

Matrix.h
Description: Matrix structure supporting basic operations like multiplication, row/column swaps, etc. Useful for solving systems of linear equations and transformations.
Time: Multiplication $\mathcal{O}(N^3)$, Row Reduction $\mathcal{O}(N^3)$.
62 lines

```
template<typename T>
struct Matrix {
```

```
    int m, n;
  vector<int> cidx;
    vector<vector<T>> entry;
    Matrix(int m, int n) : m(m), n(n), cidx(n), entry(m, vector
        <T>(n)) { iota(cidx.begin(), cidx.end(), 0); }
    Matrix(int m_) : Matrix(m_, m_) {}

    T& operator() (int r, int c) { assert(0 <= r && r < m && 0
        <= c && c < n); return entry[r][cidx[c]]; }
    const T& operator() (int r, int c) const { assert(0 <= r &&
         r < m && 0 <= c && c < n); return entry[r][cidx[c]];
        }
    Matrix operator*(const Matrix& A) const {
        assert(n == A.m);
        Matrix ret{m, A.n};
        for (int i = 0; i < ret.m; ++i) for (int k = 0; k < A.n
            ; ++k) for (int j = 0; j < ret.n; ++j)
            ret(i, j) += entry[i][k] * A(k, j);
        return ret;
    }
    void rswap(int i, int j) { swap(entry[i], entry[j]); }
    void rmul(int i, T x) { for (int j = 0; j < n; ++j) entry[i
        ][j] *= x; }
    void rdiv(int i, T x) { for (int j = 0; j < n; ++j) entry[i
        ][j] /= x; }
    void radd(int i1, int i2, T x) { for (int j = 0; j < n; ++j
        ) entry[i1][j] += entry[i2][j] * x; }
  void cswap(int i, int j) { swap(cidx[i], cidx[j]); };
  void cmul(int j, T x) { for (int i = 0; i < n; ++i) entry[i][
      cidx[j]] *= x; }
  void cadd(int j1, int j2, T x) { for (int i = 0; i < n; ++i)
      entry[i][cidx[j1]] += entry[i][cidx[j2]] * x; }
};

template<typename T>
void rref(Matrix<T>& A) {
    for (int j = 0, r = 0; j < A.n && r < A.m; ++j) {
        for (int i = r; i < A.m; ++i) {
            if (A(i, j) != 0) {
                A.rswap(r, i);
                break;
            }
        }
        if (A(r, j) != 0) {
            A.rmul(r, 1 / A(r, j));
            for (int i = 0; i < A.m; ++i) if (i != r) A.radd(i,
                 r, -A(i, r));
            ++r;
        }
    }
}

void rref(vector<ll>& A) {
  if (A.empty()) return;
    const int m = A.size();
    int sz = 60;
    for (int j = sz - 1, r = 0; j >= 0 && r < m; --j) {
        for (int i = r; i < m; ++i) {
            if ((A[i] >> j) & 1) {
                swap(A[r], A[i]);
                break;
            }
        }
        if ((A[r] >> j) & 1) {
            for (int i = 0; i < m; ++i) if (i != r) {
                if ((A[i] >> j) & 1) A[i] ^= A[r];
            }
            ++r;
        }
    }
}
```

3.2 Fast Fourier Transform

FFT.h

Description: Fast Fourier Transform (FFT) for polynomial multiplication. Supports efficient multiplication of large polynomials.
Time: $\mathcal{O}(N \log N)$, where N is the size of the input.

47 lines

```
typedef complex<double> base;

void fft(vector<base>& a, bool inv) {
    ll n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        while (!((j ^= bit) & bit)) bit >>= 1;
        if (i < j) swap(a[i], a[j]);
    }
    for (int i = 1; i < n; i <<= 1) {
        double x = inv ? M_PI / i : -M_PI / i;
        base w = {cos(x), sin(x)};
        for (int j = 0; j < n; j += i << 1) {
            base th = {1, 0};
            for (int k = 0; k < i; k++) {
                base tmp = a[i + j + k] * th;
                a[i + j + k] = a[j + k] - tmp;
                a[j + k] += tmp;
                th *= w;
            }
        }
    }
    if (inv) {
        for (int i = 0; i < n; i++) a[i] /= n;
    }
}

vector<ll> multiply(vector<ll>& A, vector<ll>& B) {
    vector<base> a(A.begin(), A.end()), b(B.begin(), B.end());

    ll n = max(a.size(), b.size());
    ll i = 0;
    while ((1 << i) < (n << 1)) i++;
    n = 1 << i;
    a.resize(n);
    b.resize(n);

    fft(a, false);
    fft(b, false);
    for (ll i = 0; i < n; i++) a[i] *= b[i];
    fft(a, true);

    vector<ll> ret;
    for (auto i: a) ret.push_back((ll) round(i.real()));
  while (ret.back() == 0) ret.pop_back();
    return ret;
}
```

FWHT.h

Description: Fast Walsh-Hadamard Transform (FWHT) for XOR, AND, OR convolutions. Supports efficient computation of subset convolutions.
Time: $\mathcal{O}(N \log N)$, where N is the size of the input.

54 lines

```
void fwht_or(vector<ll> &a, bool inv) {
    ll n = a.size();
    ll dir = inv ? -1 : 1;
    for(ll s = 2, h = 1; s <= n; s <<= 1, h <<= 1) {
        for(ll l = 0; l < n; l += s) {
            for(ll i = 0; i < h; i++) {
                a[l + h + i] += dir * a[l + i];
            }
        }
    }
}

void fwht_and(vector<ll> &a, bool inv) {
    ll n = a.size();
    ll dir = inv ? -1 : 1;
    for(ll s = 2, h = 1; s <= n; s <<= 1, h <<= 1) {
        for(ll l = 0; l < n; l += s) {
            for(ll i = 0; i < h; i++) {
                a[l + i] += dir * a[l + h + i];
            }
        }
    }
}

void fwht_xor(vector<ll> &a, bool inv) {
    ll n = a.size();
    for(ll s = 2, h = 1; s <= n; s <<= 1, h <<= 1) {
        for(ll l = 0; l < n; l += s) {
            for(ll i = 0; i < h; i++) {
                ll t = a[l + h + i];
                a[l + h + i] = a[l + i] - t;
                a[l + i] += t;
                if (inv) a[l + h + i] /= 2, a[l + i] /= 2;
            }
        }
    }
}

vector<ll> convolute(vector<ll>& A, vector<ll>& B) {
    vector<ll> a(A.begin(), A.end()), b(B.begin(), B.end());

    ll n = max(a.size(), b.size());
    ll i = 0;
    while ((1 << i) < (n << 1)) i++;
    n = 1 << i;
    a.resize(n);
    b.resize(n);

    fwht_xor(a, false);
    fwht_xor(b, false);
    for (ll i = 0; i < n; i++) a[i] *= b[i];
    fwht_xor(a, true);
    return a;
}
```

NTT.h

Description: Number Theoretic Transform (NTT) for polynomial multiplication modulo a prime. Supports efficient multiplication of large polynomials.
Time: $\mathcal{O}(N \log N)$, where N is the size of the input.

60 lines

```
// 119 * (2 ^ 23) + 1 = 998244353, root: 3
//   7 * (2 ^ 26) + 1 = 469762049, root: 3
//  17 * (2 ^ 27) + 1 = 3221225473, root: 5
const ll _A = 119, _B = 23, _P = _A << _B | 1, _X = 3;

ll modpow(ll a, ll b, ll mod) {
  if (b == 0) return 1ll;
  ll half = modpow(a, b / 2, mod);
  half = (half * half) % mod;
  if (b % 2 == 0) return half;
  else return (half * a) % mod;
}

void fft(vector<ll>& a, bool inv) {
    ll n = a.size();
    for (ll i = 1, j = 0; i < n; i++) {
```

```
        ll bit = n >> 1;
        while (!((j ^= bit) & bit)) bit >>= 1;
        if (i < j) swap(a[i], a[j]);
    }
    for (ll i = 1; i < n; i <<= 1) {
        ll w = modpow(inv ? modpow(_X, _P - 2, _P) : _X, _P / i
             >> 1, _P);
        for (ll j = 0; j < n; j += i << 1) {
            ll th = 1ll;
            for (ll k = 0; k < i; k++) {
                ll tmp = a[i + j + k] * th % _P;
                a[i + j + k] = a[j + k] - tmp;
        if (a[i + j + k] < 0) a[i + j + k] += _P;
                a[j + k] += tmp;
        if (a[j + k] >= _P) a[j + k] -= _P;
                th = th * w % _P;
            }
        }
    }
    if (inv) {
    ll ninv = modpow(n, _P - 2, _P);
        for (ll i = 0; i < n; i++) a[i] = a[i] * ninv % _P;
    }
}

vector<ll> multiply(vector<ll>& A, vector<ll>& B) {
    vector<ll> a(A.begin(), A.end()), b(B.begin(), B.end());

    ll n = max(a.size(), b.size());
    ll i = 0;
    while ((1 << i) < (n << 1)) i++;
    n = 1 << i;
    a.resize(n);
    b.resize(n);

    fft(a, false);
    fft(b, false);
    for (ll i = 0; i < n; i++) a[i] = a[i] * b[i] % _P;

    fft(a, true);
    vector<ll> ret;
  for (auto i: a) ret.push_back(i);
  while (!ret.empty() && ret.back() == 0) ret.pop_back();
    return ret;
}
```

Karatsuba.h

Description: Karatsuba Multiplication for fast polynomial multiplication. Recursive divide-and-conquer approach.
Time: $\mathcal{O}\left(N^{\log_2 3}\right)$, where N is the size of the input.

59 lines

```
const ll MOD = 998244353;

vector<ll> basic_multiply_vec(const vector<ll> &x, const vector
     <ll> &y) {
  vector<ll> ret(x.size() + y.size() + 1, 0);
  for (ll i = 0; i < x.size(); ++i) {
    for (ll j = 0; j < y.size(); ++j) {
      ret[i + j] += (x[i] * y[j]) % MOD;
      ret[i + j] %= MOD;
    }
  }
  return ret;
}

vector<ll> add(const vector<ll> &x, const vector<ll> &y, ll
     jump, ll times) {
  ll sz = max((ll)x.size(), (ll)y.size() + jump) + 2;
  vector<ll> ret = vector<ll>(x.begin(), x.end());
  while (ret.size() < sz) ret.push_back(0);

  for (ll i = 0; i < y.size(); ++i) {
    ret[i + jump] += (y[i] * times) % MOD;
    if (ret[i + jump] < 0) ret[i + jump] += MOD;
    ret[i + jump] %= MOD;
  }
  return ret;
}

vector<ll> karatsuba_multiply_vec(const vector<ll> &x, const
     vector<ll> &y) {
  ll sz = max(x.size(), y.size());
  if (sz <= 100) return basic_multiply_vec(x, y);
  ll m = sz / 2;

  // x = x0 + x1 * (10 ** m)
  vector<ll> x0 = vector<ll>(x.begin(), x.begin() + min((ll)x.
       size(), m));
  vector<ll> x1 = vector<ll>(x.begin() + min((ll)x.size(), m),
       x.end());

  // y = y0 + y1 * (10 ** m)
  vector<ll> y0 = vector<ll>(y.begin(), y.begin() + min((ll)y.
       size(), m));
  vector<ll> y1 = vector<ll>(y.begin() + min((ll)y.size(), m),
       y.end());

  // z2 = x1 * y1
  vector<ll> z2 = karatsuba_multiply_vec(x1, y1);

  // z0 = x0 * y0
  vector<ll> z0 = karatsuba_multiply_vec(x0, y0);

  // z1 = (x0 + x1) * (y0 + y1)
  x0 = add(x0, x1, 0, 1);
  y0 = add(y0, y1, 0, 1);
  vector<ll> z1 = karatsuba_multiply_vec(x0, y0);

  // z1 -= (z2 + z0) = x1 * y0 + y1 * x0
  z1 = add(z1, z2, 0, -1);
  z1 = add(z1, z0, 0, -1);

  // z2 * (10 ** 2m) + z1 * (10 ** m) + z0
  z0 = add(z0, z2, 2 * m, 1);
  z0 = add(z0, z1, m, 1);
  return z0;
}
```

3.3 Number Theory

ExtendedEucildean.h

Description: Extended Euclidean Algorithm to find integers x and y such that ax + by = gcd(a, b). Returns gcd(a, b) along with x and y.
Time: $\mathcal{O}\left(\log \min(a, b)\right)$.

5 lines

```
tuple<ll, ll, ll> extended_euclidean(ll a, ll b) {
    if (b == 0) return {a, 1, 0};
    auto [g, x, y] = extended_euclidean(b, a % b);
    return {g, y, x - (a / b) * y};
}
```

CRT.h

Description: Chinese Remainder Theorem.
crt(a, m, b, n) computes x such that $x \equiv a \pmod m$, $x \equiv b \pmod n$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m, n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$

7 lines

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
}
```

3.4 Primes

< 10^k	prime	# of prime
1	7	4
2	97	25
3	997	168
4	9973	1229
5	99991	9592
6	999983	78498
7	9999991	664579
8	99999989	5761455
9	999999937	50847534

3.5 Primality

MeisselLehmer.h

Description: Meissel-Lehmer Algorithm for counting primes up to a given number x. Combines sieve methods with recursive calculations for efficiency.
Time: $\mathcal{O}\left(x^{2/3}/\log x\right)$ for preprocessing, $\mathcal{O}\left(x^{1/2}\right)$ for queries.

102 lines

```
const int MAXV = 1e8+10;
const int MAXV2 = 1e8 + 1e4 + 10;
const int MAXLEAF = 8e5;

int pi[MAXV];
bool isPrime[MAXV];
vector<int> p;

void init() {
    p.push_back(1);
    p.push_back(2);
    isPrime[2] = true;
    for (int i = 3; i < MAXV; i += 2) isPrime[i] = true;
    int threshold = INT32_MAX / 3;
    for (int i = 3; i < MAXV; i += 2) if (isPrime[i] && i <=
         threshold) {
        for (int j = i * 3; j < MAXV; j += (i << 1)) isPrime[j]
             = false;
        p.push_back(i);
    }
    for (int i = 2; i < MAXV; ++i) pi[i] = pi[i - 1] + isPrime[
         i];
}

struct Node {
    ll n;
    int b;
    int sgn;
};

int sz;
bool arr[MAXV2];

int tree[MAXLEAF];
Node leaf[MAXLEAF];
```

```
void update(int i, int dif) { while (i < sz) { tree[i] += dif;
     i += (i & -i); } }
int query(int i) { int ret = 0; while (i > 0) { ret += tree[i];
      i -= (i & -i); } return ret; }

ll phi(const ll x, const int a) {
    ll ret = 0;
    int c = cbrt(0.5 + x);
    for (int i = 1; i <= x / c; ++i) arr[i] = false;
    int leaf_sz = 0;
    int mxx = INT32_MIN;
    vector<ll> limn;
    queue<Node> q;
    q.push(Node {1ll, a, 1});
    vector<int> v;
    while (!q.empty()) {
        Node cur = q.front(); q.pop();
        if (cur.b == 0) {
            ret += (x / cur.n) * cur.sgn;
            continue;
        }
        if (cur.n >= c) {
            int xx = x / cur.n;
            if (xx < p[cur.b] * p[cur.b]) {
                ret += (pi[xx] - cur.b + 1) * cur.sgn;
                continue;
            }
            leaf[leaf_sz++] = Node {xx, cur.b, cur.sgn};
            mxx = max(mxx, xx);
            v.push_back(xx);
            limn.push_back(mxx);
            continue;
        }
        q.push(Node {cur.n, cur.b - 1, cur.sgn});
        q.push(Node {cur.n * p[cur.b], cur.b - 1, -cur.sgn});
    }
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    sz = v.size() + 1;
    for (int i = 1; i <= v.size(); ++i) tree[i] = 0;
    int pidx = 0, vidx = 0, dif = 0;
    for (int i = leaf_sz - 1; i >= 0; --i) {
        const Node& cur = leaf[i];
        while (pidx < cur.b) {
            ++pidx;
            vidx = 0, dif = 0;
            for (int j = p[pidx]; j <= limn[i]; j += p[pidx])
                 if (!arr[j]) {
                arr[j] = true;
                if (v[vidx] < j && dif) {
                    update(vidx + 1, dif);
                    dif = 0;
                }
                while (v[vidx] < j) ++vidx;
                ++dif;
            }
            if (dif) update(vidx + 1, dif);
        }
        int idx = lower_bound(v.begin(), v.end(), cur.n) - v.
             begin();
        ret += (cur.n - query(idx + 1)) * cur.sgn;
    }
    return ret;
}

ll getPi(ll x) {
    if (x < MAXV) return pi[x];
    int c = cbrt(0.5 + x), s = sqrt(0.5 + x);
    int a = pi[c];
    ll ret = phi(x, a) + a - 1;
    for (int i = a + 1; i <= pi[s]; ++i) ret -= getPi(x / p[i])
          - i + 1;
    return ret;
}
```

MillerRab.h

Description: Miller-Rabin Primality Test for checking if a number is prime. Probabilistic test with high accuracy for large numbers.
Time: $\mathcal{O}\left(K \log^3 N\right)$, where K is the number of iterations. 37 lines

```
typedef unsigned long long ull;

const vector<vector<__int128_t>> a = {
  {2, 7, 61},
  {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}
};

__int128_t pow(__int128_t a, __int128_t b, __int128_t MOD) {
  a %= MOD;
  if (b == 0) return 1;
  __int128_t half = pow(a, b / 2, MOD);
  half = (half * half) % MOD;
  if (b % 2 == 0) return half;
  else return (half * a) % MOD;
}

bool millerRabin(__int128_t n) {
  int type = (n > INT32_MAX);
  if (type == 1 && n < 40) {
    for (auto i: a[type]) if (i == n) return true;
    return false;
  }
  for (int i = 0; i < a[type].size(); ++i) {
    if (a[type][i] == n) return true;
    __int128_t k = n - 1;
    while (true) {
      __int128_t temp = pow(a[type][i], k, n);
      if (temp == n - 1) break;
      if (k % 2 == 1) {
        if (temp == 1) break;
        else return false;
      }
      k /= 2;
    }
  }
  return true;
}
```

PollardRho.h

Description: Pollard's Rho Algorithm for integer factorization. Efficient for finding non-trivial factors of large integers. The factors returned are not sorted.
Time: $\mathcal{O}\left(N^{1/4}\right)$ for most inputs. 86 lines

```
typedef unsigned long long ull;

random_device rd;
mt19937_64 gen(rd());

const vector<vector<__int128_t>> a = {
  {2, 7, 61},
  {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}
};

__int128_t pow(__int128_t a, __int128_t b, __int128_t MOD) {
  a %= MOD;
  if (b == 0) return 1;
  __int128_t half = pow(a, b / 2, MOD);
  half = (half * half) % MOD;
  if (b % 2 == 0) return half;
  else return (half * a) % MOD;
}

bool millerRabin(__int128_t n) {
  int type = (n > INT32_MAX);
  if (type == 1 && n < 40) {
    for (auto i: a[type]) if (i == n) return true;
    return false;
  }
  for (int i = 0; i < a[type].size(); ++i) {
    if (a[type][i] == n) return true;
    __int128_t k = n - 1;
    while (true) {
      __int128_t temp = pow(a[type][i], k, n);
      if (temp == n - 1) break;
      if (k % 2 == 1) {
        if (temp == 1) break;
        else return false;
      }
      k /= 2;
    }
  }
  return true;
}

__int128_t gcd(__int128_t a, __int128_t b) {
  if (a < b) swap(a, b);
  __int128_t remainder;
  while (b != 0) {
    remainder = a % b;
    a = b;
    b = remainder;
  }
  return a;
}

__int128_t f(__int128_t x, __int128_t c, __int128_t MOD) {
     return (x * x + c) % MOD; }

void pollardRho(__int128_t n, vector<ll>& ret) {
  if (n == 1) return;
  if (n % 2 == 0) {
    ret.push_back(2);
    pollardRho(n / 2, ret);
    return;
  }
  if (millerRabin(n)) {
    ret.push_back(n);
    return;
  }
  __int128_t a, b, c, g = n;
  do {
    if (g == n) {
      uniform_int_distribution<> dist1(2, n - 1);
      a = dist1(gen); b = a;
      uniform_int_distribution<> dist2(1, 20);
      c = dist2(gen);
    }
    a = f(a, c, n);
    b = f(f(b, c, n), c, n);
    g = gcd(a > b ? a - b : b - a, n);
  } while (g == 1);
  pollardRho(g, ret);
  pollardRho(n / g, ret);
}

vector<ll> factorization(__int128_t n) {
```

```
  vector<ll> ret;
  pollardRho(n, ret);
  return ret;
}
```

DP Optimization (4)

4.1 Convex Hull Trick

점화식이 다음과 같은 형태를 만족할 때,

$$D(i) = \min_{j<i}(D(j) + B(j) \times A(i))$$

이를 직선 $l_j : y = D(j) + B(j)x$ 들을 삽입한 상황에서, $x = A(i)$를 쿼리하여 최솟값을 찾는 문제로 본다. $O(n^2) \rightarrow O(n \log n)$ (Linecontainer, Li-Chao Tree), 추가로 $B(j)$ 가 단조성을 갖는 경우 $O(n)$.

CHT.h

Description: insertion queries come in decreasing v.y/v.x order, queries come in increasing x order.

22 lines

```
const int MEM=50003;
struct cht{
    int s=0,e=0,id[MEM];
    pll f[MEM];
    double cross(int a,int b){
        return 1.0*(f[a].y-f[b].y)/(f[b].x-f[a].x);
    }
    void insert(pll v,int l){
        f[e]=v;
        id[e]=l;
        while(s+1<e && cross(e-2,e-1)>cross(e-1,e)){
            f[e-1]=f[e];
            id[e-1]=id[e];
            --e;
        }
        ++e;
    }
    ll query(ll x){
        while(s+1<e && f[s+1].y-f[s].y <= x*(f[s].x-f[s+1].x))
            ++s;
        return f[s].x * x + f[s].y;
    }
} CHT;
```

LiChaoTree.h

Description: Convex hull trick. Current implementation is for max query. Be especially aware of overflow. Let M be maximum x coordinate, $aM + b$ should be less than LLMAX.

Time: $\mathcal{O}(\log N)$

87 lines

```
struct LiChao
{
  struct Line // Linear function ax + b
  {
    int a, b;
    int eval(int x)
    {
      return a*x + b;
    }
  };
  struct Node // [start, end] has line f
  {
    int left, right;
    int start, end;
    Line f;
  };

  Node new_node(int a, int b)
  {
    return {-1,-1,a,b,{0,-INF}};
    // for min, change -INF to INF
  }

  vector <Node> nodes;

  void init(int min_x, int max_x)
  {
    nodes.push_back(new_node(min_x, max_x));
  }
  void insert(int n, Line new_line)
  {
    int xl = nodes[n].start, xr = nodes[n].end;
    int xm = (xl + xr)/2;
    Line llo, lhi;
    llo = nodes[n].f, lhi = new_line;
    if (llo.eval(xl) >= lhi.eval(xl))
      swap(llo, lhi);
    if (llo.eval(xr) <= lhi.eval(xr))
    {
      nodes[n].f = lhi;
      // for min, lhi -> llo
      return;
    }
    else if (llo.eval(xm) > lhi.eval(xm))
    {
      nodes[n].f = llo;
      // for min, llo -> lhi
      if (nodes[n].left == -1)
      {
        nodes[n].left = nodes.size();
        nodes.push_back(new_node(xl,xm));
      }
      insert(nodes[n].left, lhi);
      // for min, lhi -> llo
    }
    else
    {
      nodes[n].f = lhi;
      // for min, lhi -> llo
      if (nodes[n].right == -1)
      {
        nodes[n].right = nodes.size();
        nodes.push_back(new_node(xm+1,xr));
      }
      insert(nodes[n].right,llo);
      // for min, llo -> lhi
    }
  }
  void insert(Line f)
  {
    insert(0, f);
  }
  int get(int n, int q)
  {
    // for min, max -> min, -INF -> INF
    if (n == -1) return -INF;
    int xl = nodes[n].start, xr = nodes[n].end;
    int xm = (xl + xr)/2;
    if (q > xm)
      return max(nodes[n].f.eval(q), get(nodes[n].right, q));
    else return max(nodes[n].f.eval(q), get(nodes[n].left, q));
  }
  int get(int pt)
  {
    return get(0, pt);
  }
};
```

LineContainer.h

Description: Container where you can add lines of the form kx+m, and query maximum values at points x. Useful for dynamic programming ("convex hull trick").

Time: $\mathcal{O}(\log N)$

29 lines

```
// y = ax + b
struct Line {
  mutable ll a, b;
    mutable double p;
  bool operator< (const Line& o) const { return a < o.a; }
  bool operator< (const ll x) const { return p < x; }
};

// maximum
struct LineContainer : multiset<Line, less<>> {
  double div(ll a, ll b) { return (double) a / b; }
  bool bad(iterator x, iterator y) {
    if (y == end()) return x->p = LLONG_MAX, false;
    if (x->a == y->a) x->p = x->b > y->b ? LLONG_MAX :
        LLONG_MIN;
    else x->p = div(y->b - x->b, x->a - y->a);
    return x->p >= y->p;
  }
  void add(ll a, ll b) {
    auto z = insert({a, b, 0}), y = z++, x = y;
    while (bad(y, z)) z = erase(z);
    if (x != begin() && bad(--x, y)) bad(x, erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p) bad(x, erase
        (y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.a * x + l.b;
  }
};
```

Kitamasa.h

Description: Kitamasa's Algorithm for efficiently computing terms in a linear recurrence relation. Computes the n-th term modulo a given number. The recurrence relation coefficients $c_1, c_2, \ldots, c_m$ must be provided, and the modulo must be a prime number.

Time: $\mathcal{O}\left(M^2 \log N\right)$, where M is the order of the recurrence and N is the term to compute.

25 lines

```
const ll MOD = 1999; // Modulo value

ll a[1001]; // Initial terms of the recurrence relation
ll c[1001]; // Coefficients of the recurrence relation
ll d[1001]; // Temporary array for storing intermediate results
ll t[2001]; // Temporary array for polynomial multiplication

void kitamasa(ll n, int m) {
    if (n == 1) {
        d[1] = 1;
        for (int i = 2; i <= m; ++i) d[i] = 0;
        return;
    }
    if (n & 1) {
        kitamasa(n ^ 1, m);
        ll tmp = d[m];
        for (int i = m; i >= 1; --i) d[i] = (d[i - 1] + c[m - i
            + 1] * tmp) % MOD;
        return;
    }
    kitamasa(n >> 1, m);
    for (int i = 1; i <= 2 * m; ++i) t[i] = 0;
    for (int i = 1; i <= m; ++i) for (int j = 1; j <= m; ++j) t
        [i + j] = (t[i + j] + d[i] * d[j]) % MOD;
```

```
    for (int i = 2 * m; i > m; --i) for (int j = 1; j <= m; ++j
        ) t[i - j] = (t[i - j] + c[j] * t[i]) % MOD;
    for (int i = 1; i <= m; ++i) d[i] = t[i];
}
```

SOS.h

Description: Find $F[mask] = \sum_{i \in mask} A[i]$ in $O(N * 2^N)$. 3 lines

```
for(int i = 0; i < (1 << N); i++) F[i] = A[i];
for(int j = 0; j < N; j++) for(int i = 0; i < (1 << N); i++)
  if(i & (1 << j)) F[i] += F[i ^ (1 << j)];
```

SubmaskEnum.h

Description: Iterating through all masks with their submasks. Complexity $O(3^n)$ 3 lines

```
for (int m=0; m<(1<<n); ++m)
    for (int s=m; s; s=(s-1)&m)
        (... s and m ...)
```

4.2 Divide and Conquer Optimization

C가 Monge Array, 즉 $C(a,c) + C(b,d) \leq C(a,d) + C(b,c)$ 이고,
점화식이

$$D(i,j) = \min_{k<j}(D(i-1,k) + C(k,j))$$

형태를 만족할 때, j값에 따라 최적의 k인 opt_j가 단조증가함을 관찰하여 확인할 후보를 줄인다. $D(k, s \dots e)$ 를 구하기 위해, (1) 중간값 m에 대해 $D(k,m)$ 을 구한다. (2) 각각 재귀적으로 좌우를 호출하되, 봐야 할 j의 범위를 호출과정에서 관리한다.

DivideAndConquerDP.h

Description: Given $a[i] = \min_{lo(i) \leq k < hi(i)}(f(i,k))$ where the (minimal) optimal k increases with i, computes $a[i]$ for $i = L..R-1$.
Time: $\mathcal{O}\left((N + (hi - lo))\log N\right)$ 39 lines

```
int MAX_M, MAX_N;
vector<ll> dp_before, dp_cur;

ll C(int i, int j);

// min version
// compute dp_cur[l], ... dp_cur[r] (inclusive)
void compute(int l, int r, int optl, int optr) {
    if (l > r)
        return;

    int mid = (l + r) >> 1;
    pair<ll, int> best = {LLONG_MAX, -1};

    for (int k = optl; k <= min(mid, optr); k++) {
        best = min(best, {(k ? dp_before[k - 1] : 0) + C(k, mid
            ), k});
    }

    dp_cur[mid] = best.first;
    int opt = best.second;

    compute(l, mid - 1, optl, opt);
    compute(mid + 1, r, opt, optr);
}

void solve() {
    dp_before.assign(MAX_N,0);
    dp_cur.assign(MAX_N,0);

    for (int i = 0; i < MAX_N; i++)
        dp_before[i] = C(0, i);

    for (int i = 1; i < MAX_M; i++) {
        compute(0, MAX_N - 1, 0, MAX_N - 1);
        dp_before = dp_cur;
    }

    cout << dp_before[MAX_N - 1];
}
```

4.3 Monotone Queue Optimization

Deque Trick 수열에서 sliding window 형태의 최솟값을 빠르게 확인해야 할 때, deque에 pair형으로 (값, 인덱스) 를 저장한다. 이때 front가 우리가 확인하고자 하는 구간을 벗어났다면 계속 pop front 하고, 뒤에 삽입할 때 삽입하려는 값보다 deque의 back이 더 크다면 이를 제거하는 형태로 deque를 monotonic하게 유지한다.

DequeTrick.h

Description: Monotone Deque-DP for sliding-window like query. 12 lines

```
dq.push_back({-1, 0});
for (int i = 0; i < n; i++) {
    while (!dq.empty() and dq.front().first < i - d) {
        dq.pop_front();
    }
    dp[i] = max(arr[i], dq.front().second + arr[i]);
    while (!dq.empty() and dq.back().second <= dp[i]) {
        dq.pop_back();
    }
    dq.push_back({i, dp[i]});
    ans = max(ans, dp[i]);
}
```

점화식 D가 $D(i) = \min_{j<i}(D(j) + C(j,i))$ 형태이고, C가 Monge array일 때, $O(n^2)$ DP를 $O(n \log n)$ 으로 줄이는 기법.

가정 : 다음을 만족하는 $\text{cross}(i,j)$ 를 찾을 수 있다.

$$\text{cross}(i,j) > k \iff D(i) + C(i,k) < D(j) + C(j,k)$$

Queue Q에 앞으로 계산할 점화식에서 답이 될 수 있는 후보들을 차례로 저장한다. 즉, $D(i...n)$ 이 답이 되는 j들. 따라서, Q의 모든 원소들이
$\text{cross}(Q_i, Q_{i+1}) < \text{cross}(Q_{i+1}, Q_{i+2})$ 를 만족하고, $\text{cross}(Q_0, Q_1) \geq i$ 를 만족하도록 한다. 이때, $D(i)$ 를 $D(Q_0) + C(Q_0, i)$를 구함으로써 해결.

각 i에 대해, $\text{cross}(Q_0, Q_1) \geq i$ 을 만족시키는 것은 Q에서 필요한 만큼 pop 하여 구할 수 있고, 원소를 Q에 삽입할 때 Cross 부등식을 유지하기 위해 CHT에서와 같이 조건이 성립할때까지 맨 뒤 원소를 제거한다. Deque DP랑 비슷한 concept.

큐의 맨 뒤 원소 두개를 x, y라 할 때, $\text{cross}(x,y) \geq \text{cross}(y,i)$ 이면 y가 최적인 위치가 없으므로 이를 계속 pop한다. Cross 한번은 이분탐색으로 $O(\log n)$ 에 찾을 수 있고, 이를 n번 호출하므로 $O(n \log n)$.

Graph (5)

5.1 Theorems

Konig's Theorem In any bipartite graph,
|max flow| = |max matching| = |min vertex cover|. Also,
V − |max indep set|.

In any graph without isolated vertices,
|min edge cover| + |max matching| = V

⇒ For bipartite graph of no isolated edge,
|min edge cover| = |max indep set|

Flow complexity When all edges are unit capacities (or more generally for all intermediate vertices, there are one in/out edge of unit capacity),
Dinic's algorithm runs in $O(E \min(V^{2/3}, E^{1/2}))$

5.2 Articulation

ArficulationEdge.h

Description: Finds all articulation edges (bridges) in an undirected graph. An articulation edge is an edge whose removal increases the number of connected components.
Time: $\mathcal{O}(V + E)$, where V is the number of vertices and E is the number of edges. 26 lines

```
const int MAX_V = 10001;
vector<int> adj[MAX_V];

int ord[MAX_V], low[MAX_V];
int cnt = 0;

void dfs(vector<pair<int, int>>& ret, int v, int p) {
    ord[v] = ++cnt;
    low[v] = ord[v];
    for (auto u: adj[v]) if (u != p) {
        if (ord[u]) low[v] = min(low[v], ord[u]);
        else {
            dfs(ret, u, v);
            if (low[u] > ord[v]) ret.push_back(make_pair(min(v,
                u), max(v, u)));
      low[v] = min(low[u], low[v]);
        }
    }
}

// V: maximum vertex + 1
// 1-index
vector<pair<int, int>> getEdge(int V) {
  vector<pair<int, int>> ret;
  for (int i = 1; i < V; ++i) if (!ord[i]) dfs(ret, i, 0);
  return ret;
}
```

ArticulationPoint.h

Description: Finds all articulation points (cut vertices) in an undirected graph. An articulation point is a vertex whose removal increases the number of connected components.
Time: $\mathcal{O}(V + E)$, where V is the number of vertices and E is the number of edges. 31 lines

```
const int MAX_V = 10001;
vector<int> adj[MAX_V];

int ord[MAX_V], low[MAX_V];
int cnt = 0;

void dfs(vector<int>& ret, int v, int p) {
    ord[v] = ++cnt;
    low[v] = ord[v];
  bool flag = false;
  int child = 0;
    for (auto u: adj[v]) if (u != p) {
        if (ord[u]) low[v] = min(low[v], ord[u]);
        else {
      ++child;
            dfs(ret, u, v);
            if (low[u] >= ord[v]) flag = true;
      low[v] = min(low[u], low[v]);
        }
    }
  if (p == 0 && child > 1) ret.push_back(v); // root
  else if (p != 0 && flag) ret.push_back(v); // not root
}

// V: maximum vertex + 1
// 1-index
vector<int> getPoint(int V) {
  vector<int> ret;
  for (int i = 1; i < V; ++i) if (!ord[i]) dfs(ret, i, 0);
  return ret;
}
```

5.3 Strongly Connected Component

SCC-Kosaraju.h

Description: Strongly Connected Components (SCC) using Kosaraju's Algorithm. Finds all SCCs in a directed graph.
Time: $\mathcal{O}(V+E)$, where V is the number of vertices and E is the number of edges.

33 lines

```
const int MAX_V = 10001;
vector<int> adj[MAX_V];

vector<int> iadj[MAX_V];
bool vis[MAX_V];

void dfs(vector<int>& ord, int v) {
  vis[v] = true;
  for (auto u: adj[v]) if (!vis[u]) dfs(ord, u);
  ord.push_back(v);
}

void idfs(vector<vector<int>>& ret, int v) {
  vis[v] = true;
  ret.back().push_back(v);
  for (auto u: iadj[v]) if (!vis[u]) idfs(ret, u);
}

// V: maximum vertex + 1
vector<vector<int>> getScc(int V) {
  const int s = 1; // 1-index
  for (int u = s; u < V; ++u) for (auto v: adj[u]) iadj[v].
      push_back(u);
  vector<int> ord;
  for (int i = s; i < V; ++i) if (!vis[i]) dfs(ord, i);
  reverse(ord.begin(), ord.end());
  for (int i = s; i < V; ++i) vis[i] = false;
  vector<vector<int>> ret;
  for (auto v: ord) if (!vis[v]) {
    ret.push_back({});
    idfs(ret, v);
  }
  return ret;
}
```

SCC-Tarjan.h

Description: Strongly Connected Components (SCC) using Tarjan's Algorithm. Finds all SCCs in a directed graph.
Time: $\mathcal{O}(V+E)$, where V is the number of vertices and E is the number of edges.

39 lines

```
const int MAX_V = 10001;
vector<int> adj[MAX_V];

int ord[MAX_V], low[MAX_V];
bool inS[MAX_V];
int cnt = 0;

void dfs(vector<vector<int>>& ret, stack<int>& s, int v) {
    ord[v] = ++cnt;
    low[v] = ord[v];
    s.push(v);
    inS[v] = true;
    for (auto u: adj[v]) {
        if (inS[u]) low[v] = min(low[v], ord[u]);
        else if (!ord[u]) {
            dfs(ret, s, u);
      low[v] = min(low[u], low[v]);
        }
    }
    if (low[v] == ord[v]) {
        vector<int> scc;
        while (true) {
            int cur = s.top(); s.pop();
            inS[cur] = false;
            scc.push_back(cur);
            if (cur == v) break;
        }
        ret.push_back(scc);
    }
}

// V: maximum vertex + 1
// 1-index
vector<vector<int>> getScc(int V) {
  vector<vector<int>> ret;
    stack<int> s;
  for (int i = 1; i < V; ++i) if (!ord[i]) dfs(ret, s, i);
  return ret;
}
```

5.4 Biconnected Component

BCC.h

Description: Finds all biconnected components (BCCs) in an undirected graph. A BCC is a maximal subgraph where any two vertices are connected by at least two disjoint paths.
Time: $\mathcal{O}(V+E)$, where V is the number of vertices and E is the number of edges.

60 lines

```
struct Edge {
    int v, c, rev;
};

const int MAX_V = 300001;
vector<Edge> adj[MAX_V];

int ord[MAX_V], low[MAX_V];
bool vis[MAX_V];
int cnt = 0, col = 0;
vector<int> bcc[MAX_V];

void dfs(int v, int p) {
    ord[v] = ++cnt;
    low[v] = ord[v];
    for (const auto& edge: adj[v]) if (edge.v != p) {
        int u = edge.v;
        if (ord[u]) low[v] = min(low[v], ord[u]);
        else {
            dfs(u, v);
      low[v] = min(low[u], low[v]);
        }
    }
}

void color(int v, int c, int p) {
    if (c > 0) bcc[v].push_back(c);
    vis[v] = true;
    for (auto& edge: adj[v]) {
        int u = edge.v;
        if (low[u] >= ord[v]) {
            if (!vis[u]) bcc[v].push_back(++col);

            if (edge.c == 0) {
                edge.c = col;
                adj[u][edge.rev].c = col;
            }

            if (!vis[u]) color(u, col, v);
        } else {
            if (c > 0 && edge.c == 0) {
                edge.c = c;
                adj[u][edge.rev].c = c;
            }
            if (!vis[u]) color(u, c, v);
        }
    }
}

// V: maximum vertex + 1
// 1-index
void getBcc(int V) {
  for (int i = 1; i < V; ++i) if (!ord[i]) dfs(i, 0);
    for (int i = 1; i < V; ++i) if (!vis[i]) color(i, 0, 0);
}

void addEdge(int u, int v) {
    adj[u].push_back(Edge {v, 0, (int) adj[v].size()});
    adj[v].push_back(Edge {u, 0, (int) adj[u].size() - 1});
}
```

5.5 Network Flow

BipartiteMatch.h

Description: Bipartite Matching using DFS-based augmenting path algorithm. Finds the maximum matching in a bipartite graph.
Time: $\mathcal{O}(V \cdot E)$, where V is the number of vertices and E is the number of edges.

33 lines

```
const int MAX_N = 501;
const int MAX_M = 501;

int aMatch[MAX_N], bMatch[MAX_M];
bool vis[MAX_N];
vector<int> adj[MAX_N];

bool dfs(int a) {
  if (vis[a]) return false;
  vis[a] = true;
  for (auto b: adj[a]) {
    if (bMatch[b] == -1 || dfs(bMatch[b])) {
      aMatch[a] = b;
      bMatch[b] = a;
      return true;
    }
  }
  return false;
}

int bipartiteMatch(int N, int M) {
  for (int i = 0; i < N; ++i) aMatch[i] = -1;
  for (int i = 0; i < M; ++i) bMatch[i] = -1;
  int size = 0;
  for (int i = 0; i < N; ++i) vis[i] = false;
  for (int start = 0; start < MAX_N; ++start) {
    if (dfs(start)) {
      ++size;
      for (int i = 0; i < N; ++i) vis[i] = false;
    }
  }
  return size;
}
```

MaximumFlow.h

Description: Maximum Flow using Dinic's Algorithm. Computes the maximum flow in a flow network.
Time: $\mathcal{O}\left(V^2 \cdot E\right)$, where V is the number of vertices and E is the number of edges.

65 lines

```
struct Edge {
  int vertex;
  ll capacity;
```

```
  int reverse;
  ll flow = 0;
};

const int MAX_V = 1001;
vector<Edge> adj[MAX_V];

bool bfs(int source, int sink, vector<int>& level) {
    for (auto& i: level) i = -1;
  level[source] = 0;
  queue<int> q;
  q.push(source);
  while (!q.empty()) {
        int here = q.front(); q.pop();
    for (Edge& edge: adj[here]) {
      int there = edge.vertex;
      if (level[there] == -1 && edge.capacity - edge.flow > 0)
          {
        q.push(there);
                level[there] = level[here] + 1;
            }
        }
    }
  return level[sink] != -1;
}

ll dfs(int here, int sink, ll amount, vector<int>& level,
     vector<int>& work) {
    if (here == sink) return amount;
  for (int& i = work[here]; i < adj[here].size(); ++i){
        Edge& edge = adj[here][i];
    int there = edge.vertex;
    if (level[there] == level[here] + 1 && edge.capacity - edge
         .flow > 0) {
            ll ret = dfs(there, sink, min(edge.capacity - edge.
                 flow, amount), level, work);
      if (ret > 0) {
                edge.flow += ret;
        adj[there][edge.reverse].flow -= ret;
        return ret;
            }
        }
    }
  return 0;
}

// V: maximum vertex + 1
ll networkFlow(int source, int sink, int V) {
  for (int i = 0; i < V; ++i) for (Edge& j: adj[i]) j.flow = 0;
  ll totalFlow = 0;
  vector<int> level(V), work(V);
    while (bfs(source, sink, level)) {
        for (auto& i: work) i = 0;
    while (true) {
            ll amount = dfs(source, sink, INT32_MAX, level,
                 work);
            if (amount == 0) break;
            totalFlow += amount;
        }
    }
  return totalFlow;
}

void addEdge(int s, int e, ll c) {
  adj[s].push_back({e, c, (int)adj[e].size()});
  adj[e].push_back({s, 0, (int)adj[s].size() - 1});
}
```

MCMF.h

Description: Minimum Cost Maximum Flow using SPFA-based augmenting path algorithm. Computes the maximum flow with the minimum cost in a flow network.
Time: $\mathcal{O}((V+E)\cdot F)$, where V is the number of vertices, E is the number of edges, and F is the total flow.

68 lines

```
const ll INF = 987654321;

struct Edge {
  int vertex;
  ll capacity;
  ll cost;
  int reverse;
  ll flow = 0;
};

const int MAX_V = 502;
vector<Edge> adj[MAX_V];

void spfa(vector<int>& parent, vector<int>& ind, vector<bool>&
     inQ, vector<ll>& dis, int source) {
  for (auto& i: parent) i = -1;
  for (int i = 0; i < inQ.size(); ++i) inQ[i] = false;
  for (auto& i: dis) i = INF;
  queue<int> q;
  dis[source] = 0; inQ[source] = true; q.push(source);
  parent[source] = source;
  while (!q.empty()) {
    int here = q.front(); q.pop(); inQ[here] = false;
    for (int i = 0; i < adj[here].size(); ++i) {
      Edge& edge = adj[here][i];
      int there = edge.vertex;
      if (edge.capacity - edge.flow > 0 && dis[there] > dis[
           here] + edge.cost) {
        dis[there] = dis[here] + edge.cost;
        parent[there] = here;
        ind[there] = i;
        if (!inQ[there]) {
          q.push(there);
          inQ[there] = true;
        }
      }
    }
  }
}

// V: maximum vertex + 1
pair<ll, ll> networkFlow(int source, int sink, const int V) {
  for (int i = 0; i < V; ++i) for (Edge& j: adj[i]) j.flow = 0;
  ll totalFlow = 0;
  ll totalCost = 0;
  vector<int> parent(V), ind(V);
  vector<bool> inQ(V); vector<ll> dis(V);
  while (true) {
    spfa(parent, ind, inQ, dis, source);
    if (parent[sink] == -1) break;
    ll amount = INF;
    for (int p = sink; p != source; p = parent[p]) {
      Edge& edge = adj[parent[p]][ind[p]];
      amount = min(edge.capacity - edge.flow, amount);
    }
    for (int p = sink; p != source; p = parent[p]) {
      Edge& edge = adj[parent[p]][ind[p]];
      edge.flow += amount;
      adj[edge.vertex][edge.reverse].flow -= amount;
      totalCost += amount * edge.cost;
    }
    totalFlow += amount;
  }
  return make_pair(totalFlow, totalCost);
}

void addEdge(int s, int e, ll cap, ll dis) {
  adj[s].push_back({e, cap, dis, (int)adj[e].size()});
  adj[e].push_back({s, 0, -dis, (int)adj[s].size() - 1});
}
```

L-R MaxFlow (1) MCMF solution. For edge (a→b) with bound $[l, r]$, make two edges as $(l, -1), (r-l, 0)$. MCMF forces flow to remaining -1 edges. COST : is there possible lr-flow. FLOW : actual lr-flow. (2) Dinic solution. Make new SRC and SNK. Make (SNK, SRC, INF) edge. For edge (a→b) with bound $[l, r]$, make $(a \to \text{NEWSNK}, l)$, $(\text{NEWSRC} \to b, l)$, $(a \to b, r-l)$. Is sum of maxflow NEWSRC → NEWSNK equal to sum of l? To find actual flow, run dinic again with original SRC-SNK (without resetting).

5.6 Minimum Distance

Dijkstra.h

Description: Dijkstra's Algorithm for finding the shortest paths from a source vertex to all other vertices. Assumes non-negative edge weights.
Time: $\mathcal{O}((V+E)\log V)$, where V is the number of vertices and E is the number of edges.

26 lines

```
const int MAX_V = 100001;
vector<pair<int, ll>> adj[MAX_V];

// time complexity: (V+E)logV
// V: maximum vertex + 1
vector<ll> dijkstra(int src, int V) {
  vector<ll> dist(V, LLONG_MAX);
  dist[src] = 0;
  priority_queue<pair<ll, int>> pq;
  pq.push(make_pair(0, src));
  while(!pq.empty()) {
    ll cost = -pq.top().first;
    int here = pq.top().second;
    pq.pop();
    if (dist[here] < cost) continue;
    for (int i = 0; i < adj[here].size(); ++i) {
      int there = adj[here][i].first;
      ll nextDist = cost + adj[here][i].second;
      if (dist[there] > nextDist) {
        dist[there] = nextDist;
        pq.push(make_pair(-nextDist, there));
      }
    }
  }
  return dist;
}
```

SPFA.h

Description: Shortest Path Faster Algorithm (SPFA) for finding shortest paths in a graph. Detects negative weight cycles if they exist.
Time: $\mathcal{O}(V+E)$ on average, $\mathcal{O}(VE)$ in the worst case.

<bits/stdc++.h> 27 lines

```
using namespace std;
typedef long long ll;

const int MAX_V = 100001;
vector<pair<int, ll>> adj[MAX_V];

// empty vector: negative cycle
vector<ll> spfa(int src, int V) {
  vector<ll> dist(V, LLONG_MAX), cycleCnt(V, 0);
  vector<bool> inQ(V, false);
  dist[src] = 0;
  queue<int> q;
```

```
  q.push(src); inQ[src] = true;
  while (!q.empty()) {
    int here = q.front(); q.pop(); inQ[here] = false;
    for (auto [there, cost]: adj[here]) if (dist[there] > dist[
        here] + cost) {
      dist[there] = dist[here] + cost;
      if (!inQ[there]) {
        ++cycleCnt[there];
        if (cycleCnt[there] >= V) return {};
        q.push(there);
        inQ[there] = true;
      }
    }
  }
  return dist;
}
```

5.7 Tree

Centriod.h

Description: Centroid Decomposition for tree-based divide-and-conquer. Finds the centroid of a tree and recursively decomposes the tree.
Time: $\mathcal{O}(N \log N)$, where N is the number of vertices in the tree. 35 lines

```
const int MAX_V = 500001;
vector<pair<int, ll>> adj[MAX_V];
bool vis[MAX_V];
int sz[MAX_V];

int getSize(int v, int p) {
    int& ret = sz[v]; ret = 1;
    for (auto [u, _]: adj[v]) if (u != p && !vis[u]) ret +=
        getSize(u, v);
    return ret;
}
void getCent(int v, int p, int nsz, vector<int>& ret) {
    int psz = nsz - 1;
    bool flag = true;
    for (auto [u, _]: adj[v]) if (u != p && !vis[u]) {
        if (sz[u] * 2 >= nsz) getCent(u, v, nsz, ret);
        if (sz[u] * 2 > nsz) flag = false;
        psz -= sz[u];
    }
    if (flag && psz * 2 <= nsz) ret.push_back(v);
}
vector<int> centroid(int v, int p) {
    getSize(v, p); int nsz = sz[v];
    vector<int> ret;
    getCent(v, p, nsz, ret);
    return ret;
}

int dnc(int v) {
    int ret = 0;
  int c = centroid(v, -1)[0]; vis[c] = true;
  // compute

    for (auto [u, _]: adj[c]) if (!vis[u]) ret = max(dnc(u),
        ret); // TODO
    return ret;
}
```

HLD.h

Description: Heavy-Light Decomposition for path queries and updates in trees. Decomposes a tree into heavy and light paths for efficient segment tree operations.
Time: $\mathcal{O}(N \log N)$, where N is the number of vertices in the tree. 56 lines

```
const int MAX_V = 100001;
vector<int> adj[MAX_V];

template <class T>
struct HLD {
  T seg;
  vector<int> sz, dep, par, top, in, out;
  int cnt;

    HLD() = default;
  HLD(const vector<ll>& a): cnt(0) {
    int n = a.size();
    sz.resize(n); dep.resize(n);
    par.resize(n); top.resize(n);
    in.resize(n); out.resize(n);
    for (int i = 1; i < n; ++i) if (in[i] == 0) {
      top[i] = i;
      dfs1(i, -1);
      dfs2(i, -1);
    }
    vector<ll> arr(n);
        for (int i = 1; i < n; ++i) arr[in[i]] = a[i];
        seg = T {arr};
  }
  void dfs1(int v, int p) {
    sz[v] = 1;
    for (auto& u: adj[v]) if (u != p) {
      dep[u] = dep[v] + 1;
      par[u] = v;
      dfs1(u, v);
      sz[v] += sz[u];
      if (sz[u] > sz[adj[v][0]]) swap(u, adj[v][0]);
    }
  }
  void dfs2(int v, int p) {
    in[v] = ++cnt;
    for (auto u: adj[v]) if (u != p) {
      top[u] = (u == adj[v][0] ? top[v] : u);
      dfs2(u, v);
    }
    out[v] = cnt;
  }
  void update(int v, ll val) { seg.update(in[v], val); }
  ll query(int a, int b) {
    ll ret = 0;
    while (top[a] != top[b]) {
      if (dep[top[a]] < dep[top[b]]) swap(a, b);
      int st = top[a];
      ret = seg.merge(ret, seg.query(in[st], in[a]));
      a = par[st];
    }
    if (dep[a] > dep[b]) swap(a, b);
    ret = seg.merge(ret, seg.query(in[a], in[b]));
    return ret;
  }
};
```

LCA.h

Description: Lowest Common Ancestor (LCA) using binary lifting. Finds the lowest common ancestor of two nodes in a tree.
Time: $\mathcal{O}(N \log N)$ preprocessing, $\mathcal{O}(\log N)$ per query. 55 lines

```
const int MAX_V = 100001;
vector<pair<int, ll>> adj[MAX_V];

struct LCA {
    vector<int> dep;
    vector<ll> dis;
    vector<vector<int>> f;

    LCA(int V) {
        dep.resize(V); dis.resize(V);
        int h = (int)ceil(log2(V)); ++h;
        f = vector<vector<int>>(h, vector<int>(V));
        init(V);
    }
    void dfs(int v, int p, ll d) {
        f[0][v] = p;
        if (p == -1) {
            dep[v] = 0; dis[v] = 0ll;
        } else {
            dep[v] = dep[p] + 1; dis[v] = dis[p] + d;
        }
        for (auto [u, nd]: adj[v]) if (u != p) dfs(u, v, nd);
    }
    int lca(int u, int v) {
        const int h = f.size();
        if (dep[u] < dep[v]) swap(u, v);
        int dif = dep[u] - dep[v];
        for (int i = 0; i < h; ++i) if ((dif >> i) & 1) {
            u = f[i][u];
        }
        if (u != v) {
            for (int i = h - 1; i >= 0; --i) {
                if (f[i][u] != -1 && f[i][u] != f[i][v]) {
                    u = f[i][u]; v = f[i][v];
                }
            }
            u = f[0][u];
        }
        return u;
    }
    void init(int v) {
        const int h = f.size();
        dfs(1, -1, 0ll); // TODO: root 1
        for (int i = 1; i < h; ++i) {
            for (int j = 0; j < v; ++j) {
                if (f[i - 1][j] == -1) f[i][j] = -1;
                else f[i][j] = f[i - 1][f[i - 1][j]];
            }
        }
    }
    ll getDis(int u, int v) {
        int l = lca(u, v);
        return dis[u] + dis[v] - dis[l] * 2;
    }
};
```

Geometry (6)

6.1 Point

Point.h

27 lines

```
const double EPS = 1e-9;

template<typename T>
struct Point {
  T x, y;
  explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
  bool operator< (const Point& p) const { return tie(x, y) <
      tie(p.x, p.y); }
  bool operator== (const Point& p) const { return tie(x, y) ==
      tie(p.x, p.y); }
  bool operator!= (const Point& p) const { return !(*this == p)
      ; }
  Point operator+ (const Point& p) const { return Point(x + p.x
      , y + p.y); }
  Point operator- (const Point& p) const { return Point(x - p.x
      , y - p.y); }
```

```
  Point operator* (T d) const { return Point(x * d, y * d); }
  Point operator/(T d) const { return Point(x / d, y / d); }
  friend istream& operator>> (istream& in, Point<T>& p) { in >>
      p.x >> p.y; return in; }
  friend ostream& operator<< (ostream& out, Point<T>& p) { out
      << p.x << ' ' << p.y; return out; }
  T dot(const Point& p) const { return x * p.x + y * p.y; }
  T cross(const Point& p) const { return x * p.y - y * p.x; }
  T cross(const Point& a, const Point& b) const { return (a - *
      this).cross(b - *this); }
  T dist2() const { return x * x + y * y; }
  double dist() const { return sqrt((double)dist2()); }
  double angle() const { return atan2(y, x); } // angle to x-
      axis in interval [-pi, pi]
  Point unit() const { return *this / dist(); } // makes dist()
      = 1
  Point perp() const { return Point(-y, x); } // rotates +90
      degrees
  Point normal() const { return perp().unit(); }
  // returns point rotated a radians ccw around the origin
  Point rotate(double a) const { return Point(x * cos(a) - y *
      sin(a), x * sin(a) + y * cos(a)); }
};
```

6.2 Line

Line.h

Description: Line structure with basic operations. Includes methods for intersection, distance, and segment checks.
Time: $\mathcal{O}(1)$ per operation.

42 lines

```
template<typename T>
struct Line {
  Point<T> s, e;
  explicit Line(Point<T> s = Point<T>(0, 0), Point<T> e = Point
      <T>(1, 0)) : s(s), e(e) { assert(s != e); }
  bool sinPos() const { return s.y == e.y ? s.x <= e.x : s.y <=
       e.y; } // angle in [0, pi)
  bool operator< (const Line<T>& l) const {
    return sinPos() != l.sinPos() ? sinPos() > l.sinPos() : (e
        - s).cross(l.e - l.s) >= 0;
  }
  // only direction, not segment
  bool operator== (const Line<T>& l) const { sinPos() == l.
      sinPos() && (e - s).cross(l.e - l.s) == 0; }
  Line operator+ (const Point<T>& p) const { return Line(s + p,
       e + p); }
  Line operator- (const Point<T>& p) const { return Line(s - p,
       e - p); }
  bool onSegment(const Point<ll>& p) const { return abs(p.cross
      (s, e)) == 0 && (s - p).dot(e - p) <= 0; }
  bool onSegment(const Point<double>& p) const { return abs(p.
      cross(s, e)) < EPS && (s - p).dot(e - p) < EPS; }
  double lineDist(const Point<T>& p) const { return (double)(e
      - s).cross(p - s) / (e - s).dist(); }
  double segDist(const Point<T>& p) const {
    double d = (e - s).dist2(), t = min(d, max(0.0, (double) (p
        - s).dot(e - s)));
    return ((p - s) * d - (e - s) * t).dist() / d;
  }
  pair<int, Point<T>> lineInter(const Line& l) const {
    auto d = (e - s).cross(l.e - l.s);
    if (d == 0) return {-(s.cross(e, l.s) == 0), Point<T>(0, 0)
        };
    auto p = l.s.cross(e, l.e), q = l.s.cross(l.e, s);
    return {1, (s * p + e * q) / d};
  }
  pair<int, Point<T>> segInter(const Line& l) const {
    auto sgn = [&](T x) { return (x > 0) - (x < 0); };
    auto oa = l.s.cross(l.e, s), ob = l.s.cross(l.e, e),
        oc = s.cross(e, l.s), od = s.cross(e, l.e);
    // Checks if intersection is single non-endpoint point
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
      return {1, (s * ob - e * oa) / (ob - oa)};
    set<Point<T>> spts;
    if (l.onSegment(s)) spts.insert(s);
    if (l.onSegment(e)) spts.insert(e);
    if (onSegment(l.s)) spts.insert(l.s);
    if (onSegment(l.e)) spts.insert(l.e);
    if (spts.size() == 1) return {1, *spts.begin()};
    if (spts.size() == 0) return {0, Point<T>(0, 0)};
    return {-1, Point<T>(0, 0)};
  }
};
```

HalfPlaneInterSection.h

Description: Computes the intersection of half-planes defined by a set of lines. Returns the vertices of the intersection polygon in counter-clockwise order.
Time: $\mathcal{O}(N \log N)$, where N is the number of lines.

24 lines

```
// half plane intersection(convex)
template<typename T>
vector<Point<T>> hpi(vector<Line<T>> lines) {
  // intersection of a, b is not part of c's half plane
  auto bad = [&](Line<T>& a, Line<T>& b, Line<T>& c) {
    auto v = a.lineInter(b);
    if (v.first != 1) return false;
    return c.s.cross(c.e, v.second) <= EPS;
  };
  sort(lines.begin(), lines.end());
  deque<Line<T>> dq;
  for (int i = 0; i < lines.size(); ++i) {
    while(dq.size() >= 2 && bad(dq[(int) dq.size() - 2], dq.
        back(), lines[i])) dq.pop_back();
    while(dq.size() >= 2 && bad(dq[0], dq[1], lines[i])) dq.
        pop_front();
    if (dq.size() < 2 || !bad(dq.back(), lines[i], dq[0])) dq.
        push_back(lines[i]);
  }
  vector<Point<T>> ret;
  if (dq.size() >= 3) for (int i = 0; i < dq.size(); ++i) {
    int j = (i + 1) % dq.size();
    auto v = dq[i].lineInter(dq[j]);
    if (v.first == 1) ret.push_back(v.second);
  }
  return ret;
}
```

6.3 Polygon

ConvexHull.h

Description: Computes the convex hull of a set of points. Returns the vertices of the convex hull in counter-clockwise order.
Time: $\mathcal{O}(N \log N)$, where N is the number of points.

29 lines

```
template<typename P>
vector<P> convexHull(vector<P> pts) {
  if (pts.size() <= 1) return pts;
  sort(pts.begin(), pts.end());
  P unit = pts[0];
  for (auto& i: pts) i = i - unit;
  sort(next(pts.begin()), pts.end(), [](const P& a, const P& b)
      {
    if (a.cross(b) == 0) {
      if (a.x == b.x) return a.y < b.y;
      return a.x < b.x;
    }
    return a.cross(b) > 0;
  });
  vector<P> h;
  h.push_back(pts[0]); h.push_back(pts[1]);
  for (int i = 2; i < pts.size(); ++i) {
    while (h.size() >= 2) {
      P b = h.back(); h.pop_back();
      P a = h.back();
      if (a.cross(b, pts[i]) > 0) {
        h.push_back(b);
        break;
      }
    }
    h.push_back(pts[i]);
  }
  for (auto& i: h) i = i + unit;
  return h;
}
```

PolygonArea.h

Description: Computes twice the area of a simple polygon. Returns the absolute value of the area.
Time: $\mathcal{O}(N)$, where N is the number of vertices in the polygon.

7 lines

```
template<typename T>
T polygonArea2(const vector<Point<T>>& pts) {
  assert(!pts.empty());
  T ret = pts.back().cross(pts[0]);
  for (int i = 0; i + 1 < pts.size(); ++i) ret += pts[i].cross(
      pts[i + 1]);
  return abs(ret);
}
```

InConvex.h

Description: Checks if a point is inside a convex polygon. Returns true if the point is inside, false otherwise.
Time: $\mathcal{O}(\log N)$, where N is the number of vertices in the polygon.

12 lines

```
template <typename T>
bool inConvex(const Point<T>& p, const vector<Point<T>>& pts) {
    assert(pts.size() >= 3);
    if (pts[0].cross(pts[1], p) < 0 || pts[0].cross(pts.back(),
        p) > 0) return false;
    int l = 0, r = pts.size(), mid; --r;
    while (l + 1 < r) {
      mid = (l + r) / 2;
      if (pts[0].cross(p, pts[mid]) >= 0) r = mid;
      else l = mid;
    }
    return pts[r - 1].cross(pts[r], p) > 0;
}
```

InConcave.h

Description: Checks if a point is inside a concave polygon. Returns true if the point is inside, false otherwise.
Time: $\mathcal{O}(N)$, where N is the number of vertices in the polygon.

16 lines

```
// time complexity: N
template <typename T>
bool inConcave(const Point<T>& pt, const vector<Point<T>>& pts)
    {
  vector<Line<ll>> lines;
  const int n = pts.size();
    for (int i = 0; i < n; ++i) lines.push_back(Line<ll>(pts[i
        ], pts[(i + 1) % n]));
  Line<ll> line {pt, Point<ll> {pt.x + 100000000011, pt.y +
      1}}; // non parallel
  for (auto j: lines) if (j.onSegment(pt)) return true;
  int cnt = 0;
  for (auto j: lines) {
    auto tmp = j.segInter(line).first;
    cnt += tmp;
```

```
  }
  if (cnt % 2) return true;
  else return false;
}
```

HullDiameter.h

Description: Computes the diameter of a convex hull using rotating calipers. Returns the pair of points that are farthest apart.
Time: $\mathcal{O}(N)$, where N is the number of points in the convex hull. 17 lines

```
// rotating calipers(convex)
template<typename T>
pair<Point<T>, Point<T>> hullDiameter(vector<Point<T>>& pts) {
  int n = pts.size();
  if (n == 0) return {Point<T>(0, 0), Point<T>(0, 0)};
  if (n == 1) return {pts[0], pts[0]};
  pair<T, pair<Point<T>, Point<T>>> res({0, {pts[0], pts[0]}});
  int j = 1;
  for (int i = 0; i < j; ++i) {
    while (true) {
      res = max(res, {(pts[i] - pts[j]).dist2(), {pts[i], pts[j
          ]}});
      if ((pts[(j + 1) % n] - pts[j]).cross(pts[i + 1] - pts[i
          ]) >= 0) break;
      j = (j + 1) % n;
    }
  }
  return res.second;
}
```

6.4 Circle

Circle.h

Description: Circle structure with basic operations. Includes methods to check if a point is inside the circle.
Time: $\mathcal{O}(1)$ per operation. 33 lines

```
const double EPS = 1e-9;

template<typename T>
struct Point {
  T x, y;
  explicit Point(T x = 0, T y = 0) : x(x), y(y) {}
  bool operator< (const Point& p) const { return tie(x, y) <
      tie(p.x, p.y); }
  bool operator== (const Point& p) const { return tie(x, y) ==
      tie(p.x, p.y); }
  bool operator!= (const Point& p) const { return !(*this == p)
      ; }
  Point operator+ (const Point& p) const { return Point(x + p.x
      , y + p.y); }
  Point operator- (const Point& p) const { return Point(x - p.x
      , y - p.y); }
  Point operator* (T d) const { return Point(x * d, y * d); }
  Point operator/(T d) const { return Point(x / d, y / d); }
  friend istream& operator>> (istream& in, Point<T>& p) { in >>
       p.x >> p.y; return in; }
  friend ostream& operator<< (ostream& out, Point<T>& p) { out
      << p.x << ' ' << p.y; return out; }
  T dot(const Point& p) const { return x * p.x + y * p.y; }
  T cross(const Point& p) const { return x * p.y - y * p.x; }
  T cross(const Point& a, const Point& b) const { return (a - *
      this).cross(b - *this); }
  T dist2() const { return x * x + y * y; }
  double dist() const { return sqrt((double)dist2()); }
  double angle() const { return atan2(y, x); } // angle to x-
      axis in interval [-pi, pi]
  Point unit() const { return *this / dist(); } // makes dist()
      = 1
  Point perp() const { return Point(-y, x); } // rotates +90
      degrees
  Point normal() const { return perp().unit(); }
  // returns point rotated a radians ccw around the origin
  Point rotate(double a) const { return Point(x * cos(a) - y *
      sin(a), x * sin(a) + y * cos(a)); }
};

template<typename T>
struct Circle {
  Point<T> c; T r;
  bool inCircle(const Point<T>& p) const { return (p - c).dist
      () < r * (1 + EPS); }
};
```

CircumCircleCenter.h

Description: Computes the center and radius of the circumcircle of a triangle.
Time: $\mathcal{O}(1)$. 9 lines

```
// circumcircle center
template<typename T>
Circle<T> ccCenter(const Point<T>& a, const Point<T>& b, const
    Point<T>& c) {
  Point<T> nb = c - a, nc = b - a;
  Point<T> na = a;
  Point<T> center = na + (nb * nc.dist2() - nc * nb.dist2()).
      perp() / nb.cross(nc) / 2;
  T r = (b - a).dist() * (c - b).dist() * (a - c).dist() / (
      double) abs((b - a).cross(c - a)) / 2.0;
  return {center, r};
}
```

MinimumEnclosingCenter.h

Description: Computes the minimum enclosing circle for a set of points. Returns the center and radius of the circle.
Time: $\mathcal{O}(N)$ on average. 16 lines

```
template<typename T>
Circle<T> mec(vector<Point<T>> pts) {
  shuffle(pts.begin(), pts.end(), mt19937(time(0)));
  Circle<T> c = {pts[0], 0};
  for (int i = 0; i < pts.size(); ++i) if (!c.inCircle(pts[i]))
      {
    c = {pts[i], 0};
    for (int j = 0; j < i; ++j) if (!c.inCircle(pts[j])) {
      c.c = (pts[i] + pts[j]) / 2;
      c.r = (c.c - pts[i]).dist();
      for (int k = 0; k < j; ++k) if (!c.inCircle(pts[k])) {
        c = ccCenter(pts[i], pts[j], pts[k]);
      }
    }
  }
  return c;
}
```

Strings (7)

7.1 Knuth-Morris-Pratt

KMP.h

Description: Knuth-Morris-Pratt (KMP) string matching algorithm. Finds all occurrences of a pattern string in a text string.
Time: $\mathcal{O}(N+M)$, where N is the length of the text and M is the length of the pattern. 40 lines

```
// N[...i] maximum length with same prefix and suffix
vector<int> getPartialMatch(const string& N) {
  int m = N.size();
  vector<int> pi(m, 0);
  int begin = 1, matched = 0;
  while (begin + matched < m) {
    if (N[begin + matched] == N[matched]) {
      ++matched;
      pi[begin + matched - 1] = matched;
    } else {
      if (matched == 0) ++begin;
      else {
        begin += matched - pi[matched - 1];
        matched = pi[matched - 1];
      }
    }
  }
  return pi;
}

// index of H's substring is N
vector<int> kmpSearch(const string& H, const string& N) {
  int n = H.size(), m = N.size();
  vector<int> ret;
  vector<int> pi = getPartialMatch(N);
  int begin = 0, matched = 0;
  while (begin <= n - m) {
    if (matched < m && H[begin + matched] == N[matched]) {
      ++matched;
      if (matched == m) ret.push_back(begin);
    } else {
      if (matched == 0) ++begin;
      else {
        begin += matched - pi[matched - 1];
        matched = pi[matched - 1];
      }
    }
  }
  return ret;
}
```

7.2 Palindrome

Manacher.h

Description: Manacher's Algorithm for finding all palindromic substrings. Computes the radius of the largest palindrome centered at each position.
Time: $\mathcal{O}(N)$, where N is the length of the string. 26 lines

```
vector<int> manacher(const string& s) {
  const int n = s.size();
  vector<int> ret(n);
  int r = -1, p = -1;
  for (int i = 0; i < n; ++i) {
    if (i < r) ret[i] = min(ret[2 * p - i], r - i);
    while (i - ret[i] - 1 >= 0 && i + ret[i] + 1 < n && s[i -
        ret[i] - 1] == s[i + ret[i] + 1]) ++ret[i];
    if (r < i + ret[i]) {
      r = i + ret[i];
      p = i;
    }
  }
  return ret;
}

// radius = palindrome
vector<int> palindrome(const string& S) {
  // #a#b#c#d#e#
  string ts;
  for (auto i: S) {
    ts += '#';
    ts += i;
  }
  ts += '#';
```

```
  return manacher(ts);
}
```

7.3 Trie

Trie.h

Description: Trie (prefix tree) for efficient string storage and retrieval. Supports insertion, search, and deletion of strings.
Time: $\mathcal{O}(L)$ per operation, where L is the length of the string.

40 lines

```
const int ALPHABETS = 26;
int toNumber(char ch) { return ch - 'A'; };

struct TrieNode {
  TrieNode* children[ALPHABETS];
  bool terminal;

  TrieNode(): terminal(false) { memset(children, 0, sizeof(
      children)); }
  ~TrieNode() { for (int i = 0; i < ALPHABETS; ++i) if (
      children[i]) delete children[i]; }
  void insert(const string& s, int idx = 0) {
    if (idx == s.size()) terminal = true;
    else {
      int next = toNumber(s[idx]);
      if (children[next] == nullptr) children[next] = new
          TrieNode();
      children[next]->insert(s, idx + 1);
    }
  }
  bool find(const string& s, int idx = 0) {
    if (idx == s.size()) return this->terminal;
    int next = toNumber(s[idx]);
    if (children[next] == nullptr) return false;
    return children[next]->find(s, idx + 1);
  }
  bool erase(const string& s, int idx = 0) {
    if (idx == s.size()) {
      terminal = false;
      for (int i = 0; i < ALPHABETS; ++i) if (children[i])
          return false;
      return true;
    }
    int next = toNumber(s[idx]);
    if (children[next] == nullptr) return false;
    if (children[next]->erase(s, idx + 1)) {
      delete children[next];
      children[next] = nullptr;
    }
    if (terminal) return false;
    for (int i = 0; i < ALPHABETS; ++i) if (children[i]) return
        false;
    return true;
  }
};
```

7.4 AhoCorasick

ahoCorasick.h

Description: Aho-Corasick algorithm for finding multiple patterns in a text. Builds a trie from a set of patterns and adds failure links to efficiently transition between states.
Time: $\mathcal{O}(L + N)$, where L is the sum of pattern lengths and N is the text length.

58 lines

```
const int ALPHABETS = 26;
int toNumber(char ch) { return ch - 'a'; };

struct TrieNode {
  TrieNode* children[ALPHABETS];
  TrieNode* failure;
  bool terminal;

  TrieNode(): terminal(false) { memset(children, 0, sizeof(
      children)); }
  ~TrieNode() { for (int i = 0; i < ALPHABETS; ++i) if (
      children[i]) delete children[i]; }
  void insert(const string& s, int idx = 0) {
    if (idx == s.size()) terminal = true;
    else {
      int next = toNumber(s[idx]);
      if (children[next] == nullptr) children[next] = new
          TrieNode();
      children[next]->insert(s, idx + 1);
    }
  }
  bool find(const string& s, int idx = 0) {
    if (idx == s.size()) return this->terminal;
    int next = toNumber(s[idx]);
    if (children[next] == nullptr) return false;
    return children[next]->find(s, idx + 1);
  }
};

void getFailure(TrieNode* trie) {
  queue<TrieNode*> Q;
  trie->failure = trie;
  for (int i = 0; i < ALPHABETS; ++i) if (trie->children[i]) {
    trie->children[i]->failure = trie;
    Q.push(trie->children[i]);
  }
  while(!Q.empty()) {
    TrieNode* here = Q.front(); Q.pop();
    for (int i = 0; i < ALPHABETS; ++i) if (here->children[i])
        {
      TrieNode* there = here->children[i];
      TrieNode* p = here->failure;
      while (p != trie && p->children[i] == nullptr) p = p->
          failure;
      if (p->children[i]) p = p->children[i];
      there->failure = p;
      if (p->terminal) there->terminal = true;
      Q.push(there);
    }
  }
}

// check s's substring
bool query(string& s, TrieNode* trie) {
  TrieNode* p = trie;
  for (auto& i: s) {
    int next = toNumber(i);
    while (p != trie && p->children[next] == nullptr) p = p->
        failure;
    if (p->children[next]) p = p->children[next];
    if (p->terminal) return true;
  }
  return false;
}
```

7.5 Suffix Array

SuffixArray.h

Description: Constructs the suffix array and LCP (Longest Common Prefix) array of a string. Suffix Array Time: $O(N \log N)$, where N is the length of the string. LCP Array Time: $O(N)$.

67 lines

```
bool radixSort(vector<int>& rk, int k) {
    const int n = rk.size();
    int m = *max_element(rk.begin(), rk.end());
    if (n == m) return false;
    vector<int> sr(n), srk(n);
    vector<int> cnt(m + 1);

    vector<int> rkk(n);
    for (int i = 0; i < n; ++i) {
        if (i + k < n) rkk[i] = rk[i + k];
        else rkk[i] = 0;
    }

    for (auto i: rkk) ++cnt[i];
    for (int i = 1; i <= m; ++i) cnt[i] += cnt[i - 1];
    for (int i = 0; i < n; ++i) srk[--cnt[rkk[i]]] = i;
    reverse(srk.begin(), srk.end());

    for (auto& i: cnt) i = 0;
    for (auto i: rk) ++cnt[i];
    for (int i = 1; i <= m; ++i) cnt[i] += cnt[i - 1];
    for (int i = 0; i < n; ++i) {
        int j = srk[i];
        sr[--cnt[rk[j]]] = j;
    }

    vector<int> nrk(n);
    m = 1; nrk[sr[0]] = m;
    for (int i = 1; i < n; ++i) {
        int cur = sr[i], prv = sr[i - 1];
        if (rk[cur] != rk[prv] || rkk[cur] != rkk[prv]) ++m;
        nrk[cur] = m;
    }
    swap(nrk, rk);
    return true;
}

vector<int> getSuffixArray(const string& s) {
    const int n = s.size();
    const int m = 255;
    vector<int> cnt(m);
    for (auto i: s) cnt[i] = 1;
    for (int i = 1; i < m; ++i) cnt[i] += cnt[i - 1];

    vector<int> rk(n);
    for (int i = 0; i < n; ++i) rk[i] = cnt[s[i]];
    int k = 1;
    while (radixSort(rk, k)) k *= 2;

    vector<int> sfx(n);
    for (int i = 0; i < n; ++i) sfx[rk[i] - 1] = i;
    return sfx;
}

vector<int> getLcp(const string& s, const vector<int>& sfx) {
  const int n = s.size();
  vector<int> inv(n), lcp(n);
  for (int i = 0; i < n; ++i) inv[sfx[i]] = i;
  int h = 0;
  for(int i = 0; i < n; ++i) if (inv[i]) {
    int prv = sfx[inv[i] - 1];
    while (s[prv + h] == s[i + h]) ++h;
    lcp[inv[i]] = h;
    h = max(h - 1, 0);
  }
  return lcp;
}
```

7.6 Z

Z.h

Description: Z-Algorithm for pattern matching and substring search. Computes the Z-array, where Z[i] is the length of the longest substring starting from S[i] that matches a prefix of S.

Time: $\mathcal{O}(N)$, where N is the length of the string. 13 lines

```
vector<int> getZ(const string& s) {
  const int n = s.size();
  int l = 0, r = 0;
  vector<int> z(n, 0);
  z[0] = n;
  for (int i = 1; i < n; ++i) {
    if (i <= r) z[i] = min(r - i, z[i - l]);
    while (s[i + z[i]] == s[z[i]]) ++z[i];
    if (i > r) l = i;
    r = max(r, i + z[i] - 1);
  }
  return z;
}
```